

M00 Learner Book

Absolute-beginner teacher book - synchronized to 8 canonical lessons

How this book teaches

Every lesson follows: Understand -> See it -> Do it with me -> Expected result -> Why -> Your turn -> Teach-back -> Review only after attempt.

Important install boundary

This book teaches installation discipline, but does not require early WSL/Python/PostgreSQL/Docker/Spark/Kafka/Fabric setup. Use the current module as the authority for when a tool is first installed.

M00-L01 - Files, Folders, Drives and Paths

By the end: Find a file deliberately and explain its name, containing folder and full path.

Plain-English mental model

A file name is like a person name; a path is the full address that tells the computer where the item lives.

Do it with me

Create Documents\DE_COURSE_PRACTICE\files_lab, add location_test.txt, reopen File Explorer and locate it without Recent.

Expected result

One stable course folder, one files_lab folder and one location_test.txt that remains after File Explorer is closed.

Why this matters later

Later tools fail when the learner cannot distinguish file identity from file location.

Common wrong turns

- Looking in Downloads when the file is in Documents.
- Creating project_final_final folders instead of one stable workspace.
- Using Recent instead of knowing the actual location.

Teach-back

- File vs folder?
- What is missing if I only say orders.csv?
- What does a path tell you?

Your turn - independent

Inside files_lab create practice/my_address_test.txt. Close File Explorer, reopen from Documents, find the file without Recent and write its full path inside the file.

Evidence: Screenshot or note containing the exact folder path and the full file path.

M00-L02 - File Extensions and Common Data Files

By the end: Recognize .xlsx, .csv, .json, .txt, .sql, .py, .zip and .pdf without confusing the app with the file format.

Plain-English mental model

The extension describes the file format. The app icon only shows a default way Windows may open it.

Do it with me

Turn on File name extensions. Open sample.csv first in a text editor and then in Excel if available; observe that it remains CSV.

Expected result

Extensions are visible and the same CSV can be inspected as comma-separated text and as rows/columns.

Why this matters later

Data engineering tools choose parsers from the real format, not from the icon shown in Windows.

Common wrong turns

- Renaming .csv to .xlsx and assuming conversion happened.
- Hiding extensions and creating report.csv.csv.
- Opening code files in rich-text software that can damage plain text.

Teach-back

- Which part of report.csv is the extension?
- Why can CSV open in Excel without becoming XLSX?
- Why is rename not conversion?

Your turn - independent

Choose four bundled files, predict each format from its extension before opening, then verify with a suitable app.

Evidence: A four-row note: filename, predicted format, app used, what you observed.

M00-L03 - Download, ZIP, Extract, Copy, Move and Rename

By the end: Extract archives safely and distinguish copy, move, rename and extract.

Plain-English mental model

A ZIP is a sealed travel bag. Extraction unpacks normal working copies; the ZIP can remain as the clean source.

Do it with me

Copy DOWNLOAD_ME_FIRST.zip to Downloads, Extract All, rename the extracted folder m00_download_lab and move that working folder into Documents\DE_COURSE_PRACTICE\01_MODULE_WORK\M00. Keep the original ZIP.

Expected result

Both the source ZIP and a separate editable extracted working folder exist.

Why this matters later

Projects often arrive as archives. Keeping source and working copies separate prevents accidental evidence loss.

Common wrong turns

- Editing inside the ZIP preview.
- Moving the only clean source when a copy was intended.
- Downloading duplicate copies instead of locating the first one.

Teach-back

- What does extraction do?
- When choose Copy instead of Move?
- Why keep a clean source?

Your turn - independent

Copy messy_downloads to messy_downloads_working and organize only the working copy. Keep source untouched.

Evidence: Before/after folder listing proving the source is unchanged and the working copy was reorganized.

M00-L04 - Installing Software Safely

By the end: Distinguish an installer from an installed application, use official sources, understand PATH at beginner level and verify an install.

Plain-English mental model

An installer delivers/configures an application. PATH is a search list a shell uses to find command programs.

Do it with me

Using VS Code as the example, identify the official download page, the installed Start-menu application and (if still present) the installer file. Do not install unrelated tools.

Expected result

Learner can explain installer vs application, source trust, version verification and PATH without installing the whole DE stack.

Why this matters later

Blind early installation creates unnecessary failures and version conflicts.

Common wrong turns

- Clicking sponsored download buttons.
- Approving UAC prompts without checking what launched them.
- Installing Python/PostgreSQL/WSL/Docker early "just in case".

Teach-back

- Installer vs installed app?
- Why use official source?
- What does PATH do?

Your turn - independent

Pick one application already on the PC. Identify how you launch it and, if available, the old installer. Explain why they are different.

Evidence: Short note with app name, launch location, installer location/status and verification method.

M00-L05 - VS Code From Zero

By the end: Open the correct project root, use Explorer/Editor/Tabs/Terminal, save a file to the intended folder and reopen the workspace.

Plain-English mental model

VS Code is a technical workspace/desk; it is not Python, SQL, Git or a database.

Do it with me

Open m00_download_lab as a folder, inspect data and notes, create evidence.txt at project root, save, close and reopen.

Expected result

Explorer shows the whole project tree and evidence.txt survives in the intended location.

Why this matters later

Opening the correct root makes paths, terminals and later project structure understandable.

Common wrong turns

- Opening one random file instead of the project root.
- Creating a file in the wrong selected subfolder.
- Closing an Untitled/unsaved tab.

Teach-back

- What is VS Code for?
- What is project root?
- Explorer vs Editor?

Your turn - independent

Open supplier_import_assets as project root. Create evidence/hello.txt at the correct level, close and reopen, then verify location.

Evidence: Project-tree screenshot or written path to evidence/hello.txt plus one sentence explaining project root.

M00-L06 - Terminal Fundamentals

By the end: Explain prompt/current directory/command/argument and safely use pwd, ls, cd, cd .. and mkdir.

Plain-English mental model

File Explorer uses clicks; a terminal uses typed instructions. Every command runs from a current directory.

Do it with me

From the VS Code terminal at m00_download_lab: pwd, ls, cd data, pwd, ls, cd .., mkdir terminal_created.

Expected result

Learner can predict where the new folder appears and can explain each path change.

Why this matters later

Many later DE tools are command-driven; location awareness prevents most beginner terminal mistakes.

Common wrong turns

- Running the right command from the wrong directory.
- Treating a path as if it were a command.
- Copying destructive commands without understanding them.

Teach-back

- What is current directory?
- In mkdir evidence, command vs argument?
- What does cd .. do?

Your turn - independent

From supplier_import_assets root, enter data, list files, return to root, create archive, enter it, create 2026, and record pwd at each level.

Evidence: Saved terminal transcript or screenshot sequence showing the path changes and created folders.

M00-L07 - Errors and Beginner Troubleshooting

By the end: Use one hypothesis -> one change -> one retry instead of random fixing.

Plain-English mental model

An error is evidence. Symptom, cause and fix are different things.

Do it with me

Use exercises/broken_setup/TASK.txt. Read the symptom, find the misplaced file, repair only the location problem and write repair_note.txt while preserving broken_setup_source.

Expected result

One controlled fix with a repair note that names symptom, cause, change and verification.

Why this matters later

Random multi-change troubleshooting hides the real cause and makes the next failure harder.

Common wrong turns

- Reinstalling software to fix a path problem.
- Changing several settings at once.
- Deleting the broken original evidence.

Teach-back

- Why change one thing at a time?
- For file not found, what inspect first?
- What makes a repair note useful?

Your turn - independent

Use exercises/path_problem/TASK.txt. Save attempt note first, form one hypothesis at a time, fix the path issue and write a two-sentence cause/repair explanation.

Evidence: attempt_note.txt plus repaired state plus explanation of cause and verification.

M00-L08 - Study Evidence, Attempts and Review Discipline

By the end: Separate learning examples, independent attempts, review, retests and evidence so completion reflects real skill.

Plain-English mental model

Recognition is not recall. Preserve the first attempt because it shows what you actually knew before review.

Do it with me

Create 02_ATTEMPTS, 03_REVIEW, 04_RETESTS, 05_EVIDENCE. Save m00_test_attempt.txt, then a separate m00_test_retest.txt and evidence note.

Expected result

Original attempt remains unchanged; retest is separate; evidence identifies both.

Why this matters later

Job work and interviews require recall and reasoning without an answer sheet.

Common wrong turns

- Editing the first attempt after seeing review.
- Calling Reviewed the same as Mastered.
- Calling Skipped the same as Mastered.

Teach-back

- Why save first attempt?
- Review vs retest?
- Why is skipped not mastered?

Your turn - independent

Create attempts/P05_attempt1.txt with an intentionally wrong sentence, then retests/P05_retest1.txt with correction; explain both in evidence/P05_evidence.txt.

Evidence: Three separate files proving attempt, retest and explanation.

M00 move-on standard

Move to M01 only when you can find the course workspace, identify common file formats, extract and preserve source archives, explain safe installation, open a project root in VS Code, navigate a terminal safely, troubleshoot one path problem with one hypothesis at a time, and preserve attempts/retests/evidence separately.